Deree
School of Graduate and Professional Education

Big Data Architectures
Winter 2026

Term Project

Due: Monday March 23, 2026, 11:59.

This project setups a network of financial and investment services of stock exchanges and investors as well as applications that analyze those data. The components of the network are described below.

Two Stock Exchange servers. The two servers should be implemented in files *se1_server.py* and *se2_server.py*, respectively. Each server trades 12 stocks and emits JSON objects at the close of each day. Stock Exchanges trade every day except weekends and Greek national holidays, starting 1/1/2020. The prices of each stock at the close of each day can be ±3% of the price of the previous day close. The duration of each day will be simulated with at least 3 seconds of sleep, i.e., each server will emit no earlier than every 3 seconds the closing stock prices for a date.

The first server emits its output to port 9999, while the second to Kafka topic *StockExchange*.

The sample Stock Exchange server in file *server.py* simulates the functions of a fictious Stock Exchange and emits JSON objects to port 9999 at the closing of each day. You need to update the missing parts of the code for the first server and update the code for the second server.

One server should emit the first 12 stocks of those shown in *server.py* and the other will send to Kafka topic *StockExchange* the rest.

Three Institutional Investors Inv1, Inv2, and Inv3. Each investor manages two portfolios as follows: Inv1 manages P11 and P12, Inv2 manages P21 and P22, and Inv3 manages P31 and P32. The portfolios are structured as follows:

Inv1			Inv2			Inv3		
Portfolio	Stock	Qty	Portfolio	Stock	Qty	Portfolio	Stock	Qty
P11	IBM	13000	P21	HPQ	16000	P31	HPQ	22000
	AAPL	22000		CSCO	17000		ZM	18000
	META	19000		ZM	19000		DELL	24000
	AMZN	25000		QCOM	21000		NVDA	12000
	GOOG	19000		ADBE	28000		IBM	19000
	AVGO	24000		VZ	17000		INTC	16000
P12	VZ	29000	P22	TXN	14000	P32	VZ	18000
	INTC	26000		CRM	26000		AVGO	29000
	AMD	21000		AVGO	17000		NVDA	16000
	MSFT	12000		NVDA	18000		AAPL	22000
	PLTR	27000		MSTR	26000		DELL	25000
	ORCL	12000		MSI	18000		ORCL	20000

Each investor reads the JSON objects from the Stock Exchange Servers and writes to Kafka topic *portfolios* a message with key the portfolio name and value a JSON object that contains

a. the Net Asset Value (NAV) of the portfolio for each day,
b. difference from the previous day's NAV evaluation,
c. percentage difference from previous day's NAV evaluation.

NAV is calculated as

(total evaluation of assets – liabilities) / number of shares

where

- Total evaluation of assets is the value of the portfolio at the close of each day.
- Labilities is simulated to be a random amount from 65% to 70% of the total evaluation of assets.

The investors should be implemented in files *inv1.py*, *inv2.py*, *inv3.py*, respectively.

An application, which creates a MySQL database 'InvestorsDB' with the following tables

1. 'Investors' with fields 'Id', 'Name', 'City'.
2. 'Portfolios' with fields 'Id, 'Name', 'Cumulative'.
3. 'Investors_Portfolios' with fields 'iid', 'pid' that connects investors and portfolios.
4. <investor>_<portfolio> per investor and portfolio, e.g., Inv1_P11, Inv12_P12, and so on. The content of each file should be similar to the following.

Date	NAV per Share	Daily NAV Change	Daily NAV Change %
03-Mar-23	51.47	1.25	2.49
02-Mar-23	50.22	–0.44	–0.87
01-Mar-23	50.66	0.06	0.12
28-Feb-23	50.6	–0.43	–0.84
27-Feb-23	51.03	0.39	0.77

and initializes the first three tables with the data shown above. The application should be implemented in file *investorsDB.py*.

An application that reads the Kafka topic *portfolios* and adds data to the *InvestorsDB* database, as new JSON objects are read from the topic. The application should be implemented in file *app1.py*.

A Spark DF application that queries the *InvestorsDB* database and for a given investor (provided as a command line argument) produces a file for each of its portfolios with name <investor>_<portfolio>_stats that contains the following.

- The maximum and minimum daily change and percentage change for the full history of the portfolio.

- The maximum and minimum daily change and percentage change for the full history of the portfolio per year.
- The average evaluation and the standard deviation of evaluation for the full history of the portfolio.
- The average evaluation and standard deviation of the portfolio for a given period, e.g., 2020 to 2024.
- The average evaluation per month for the full history of the portfolio, with the most recent month listed first.

The application should be implemented in file *app2.py*.

An application that reads the Kafka topic *portfolios* and adds data to a MongoDB database called *Portfolios* that contains a separate collection for each portfolio. The application should be implemented in file *app3.py*.

An application that reads data from the MongoDB database and for a given portfolio and time period (provided as command line arguments) creates a report on the output the contains:

1. The name of the portfolio
2. The start and end time of the given period.
3. The number of NAV values and the min, max and average NAV values in the given period.

The application should be implemented in file *app4.py*.

Big Data Architectures
Winter 2026

Submission

Each team should submit

1. All Python code.
2. A Powerpoint presentation containing
    a. A short description of the problem.
    b. The architecture of the solution.
    c. The design decisions made.
    d. Screenshots of the running application.
    e. Lessons learnt.

    The presentation should not be more than 15 slides excluding the first one that should contains the name and logo of the team and the names of its members, and the last one that should thank and prompt for questions.
3. A file Team.txt containing a clear statement stating which parts of the application where implemented by which team member and a self-assessment of the percentage of contribution of each team member to the project (e.g., Efthimiou 45%, Papadopoulou 55%).
4. A README.txt file with instructions on how to run the network of applications.

To submit your work add

- all source code files,
- the Powerpoint presentation,
- the file Team.txt
- the file README.txt

into a rar or zip compressed file with name <Lastname>_<Lastname>.rar or .zip (e.g., Efthimiou_Papadopoulou.zip) and submit it to Blackboard. Names must appear in alphabetical order with first letter in capital and the rest in small case.

Penalties

Violation of any naming conventions will result in 30 points reduced from your grade.

Submission of unnecessary files (anything except the .py files above) will result in 20 points reduced from your grade.